

```
vhdl

sram_model : process(clk)
begin
  if rising_edge(clk) then
    if sram_ce1 = '0' then
      if sram_wen = '1' then
        -- WRITE: wen=1 means write
        sram_mem(to_integer(
          unsigned(sram_addr(11 downto 2)))) <= sram_wdata;
        sram_rdata <= (others => '0');
      elsif sram_wen = '0' and sram_oen = '0' then
        -- READ: wen=0 and oen=0 means read
        if unsigned(sram_addr) >= 16#F00000# then
          sram_rdata <= x"DEAD0000";
        else
          sram_rdata <= sram_mem(to_integer(
            unsigned(sram_addr(11 downto 2))));
        end if;
      else
        sram_rdata <= (others => '0');
      end if;
    else
      sram_rdata <= (others => '0');
    end if;
  end if;
end process;
```

Comparison of All 3 Versions

Version	Write condition	Read condition
Your working model	ce1=0, oen=1	ce1=0, oen=0
Active low wen	ce1=0, wen=0	ce1=0, wen=1, oen=0
Active high wen (this one)	ce1=0, wen=1	ce1=0, wen=0, oen=0

active low version is crct and also during INCR(4) and onwards test is showing this..the write is showing up in my sram_wdata but not writing it

properly in sram mem (because the first addr changes before oen and wen and cen values are set to their values) and also during the the read for all the burst tests. Eg: in INCR4 , IN SRAM_RDATA ONLY THE 1ST AND 3RD IS SEEN. AND IN INCR8 ONLY 1ST AND THEN 0 THEN LAST ONE IS SEEN. THE ADDRESS IS HAPPENING PROPERLY IN HRADDR AND IN SRAM_ADDR WITHIN THE EXPECTED RANGE 8 PULSES . My assumption is that read needs two cycles to execute it..we might need to change the sram model to compensate it.

You are exactly right! This is a classic **pipeline timing issue** in AHB.

The Problem

AHB is pipelined — **address comes 1 cycle before data/control signals**:

Clock:	1	2	3	4	5
haddr:	A0	A1	A2	A3	IDLE
htrans:	NONSEQ	SEQ	SEQ	SEQ	IDLE
sram_addr:	A0	A1	A2	A3	
sram_oen:	X	0	0	0	← lags by 1 cycle!
sram_wen:	X	0	0	0	← lags by 1 cycle!
sram_ce1:	X	0	0	0	← lags by 1 cycle!

So when `sram_addr=A0`, the `oen/wen/ce1` are not yet valid. They become valid only on the **next cycle** when `sram_addr=A1`. This is why you see only some beats.

The Fix

The SRAM model needs to **register the address one cycle** and use the registered address when control signals are valid:

```
vhdl

sram_model : process(clk)
    variable registered_addr : std_logic_vector(ABITS-1 downto 0);
begin
    if rising_edge(clk) then

        -- Stage 1: Register address every cycle
        -- (address is valid 1 cycle before control signals)
        registered_addr := sram_addr;

        -- Stage 2: Use registered address with current control signals
```

```
if sram_ce1 = '0' then
  if sram_oen = '1' then
    -- WRITE: use registered address
    sram_mem(to_integer(
      unsigned(registered_addr(11 downto 2)))) <= sram_wdata;
    sram_rdata <= (others => '0');

  elsif sram_oen = '0' then
    -- READ: use registered address
    if unsigned(registered_addr) >= 16#F000000# then
      sram_rdata <= x"DEAD0000";
    else
      sram_rdata <= sram_mem(to_integer(
        unsigned(registered_addr(11 downto 2))));
    end if;
  end if;
else
  sram_rdata <= (others => '0');
end if;

end if;
end process;
```

Why This Works

Clock:	1	2	3	4	
sram_addr:	A0	A1	A2	A3	
reg_addr:	--	A0	A1	A2	← registered = 1 cycle delayed
sram_oen:	X	0	0	0	← valid from cycle 2
sram_ce1:	X	0	0	0	

```
-- Now when ce1=0 and oen=0 at cycle 2:
--   registered_addr = A0  ← CORRECT address for this data beat!
-- At cycle 3:
--   registered_addr = A1  ← CORRECT!
-- And so on...
```

What You Should See After Fix

Test	Before	After
INCR4 Read	beat 1 and 3 only	all 4 beats correct

Test	Before	After
INCR8 Read	beat 1, 0, last	all 8 beats correct
WRAP4/8/16	partial beats	all beats correct
SINGLE	correct already	still correct

What did you change with respect to read? The clk was the issue? Was it no?

Yes exactly! The clock was the issue.

What Changed

Old model was combinational:

```
vhdl

sram_model : process(sram_ce1, sram_oen, sram_addr)
-- sensitivity list = signals, not clk
-- reacts INSTANTLY when sram_addr changes
```

So when `sram_addr` changed to `A1`, the model immediately tried to read using `A1` but `oen/ce1` hadn't updated yet — **wrong timing**.

New model is clocked:

```
vhdl

sram_model : process(clk)
-- only reacts on rising edge of clk
-- address is registered (delayed by 1 cycle)
```

So:

```
Cycle 1: sram_addr=A0 arrives → registered_addr captures A0
Cycle 2: oen=0, ce1=0 valid → reads using registered_addr=A0 ✓
```

Cycle 2: sram_addr=A1 arrives → registered_addr captures A1
 Cycle 3: oen=0, ce1=0 valid → reads using registered_addr=A1 ✓

One Line Summary

The combinational model reacted to address changes **immediately** before control signals were ready. The clocked model **waits one cycle** so address and control signals are always in sync.

The oen and wen and cen not happening was a issue only for the write is what I thought..because those signals get their cect values 2 cycles before itself but then the actual values only show up in the middle of those signals

Yes you are right! Let me break it down clearly.

Write Issue

Clock:	1	2	3	4	
sram_addr:	A0	A1	A2	A3	
sram_ce1:	X	0	0	0	← valid from cycle 2
sram_wen:	X	0	0	0	← valid from cycle 2
sram_wdata:	X	D0	D1	D2	← valid from cycle 2

So at cycle 1, `sram_addr=A0` but `ce1/wen/wdata` are all invalid. The write for `A0` only happens at cycle 2 when control signals are valid — but by then `sram_addr` has already moved to `A1`. So **A0's data gets written to A1's location** — wrong address.

The registered address fixes this by using `A0` (registered) when `ce1/wen` become valid at cycle 2. ✓

Read Issue

Clock:	1	2	3	4	5
sram_addr:	A0	A1	A2	A3	IDLE
sram_oen:	X	0	0	0	
sram_rdata	needs to be valid for hrdata at cycle 2,3,4				

The read issue is **different** — it's not about control signals being late. It's about `sram_rdata` needing **2 cycles** to come back:

Cycle 2: `oen=0`, `addr=A1` → `sram_rdata` starts being fetched
Cycle 3: `sram_rdata=D0` valid → `hrdata` captures it

So the AHB bridge expects `sram_rdata` to be valid **the cycle after** it asserts `oen`. The combinational model was trying to respond in the **same cycle** which was too fast and caused misalignment.

Bottom Line

Issue Root Cause

Write Address moves to A1 before `ce1/wen` valid for A0

Read `sram_rdata` needs 1 extra cycle to be valid — bridge samples it the cycle after `oen` asserted

Both are fixed by the clocked+registered model because it naturally introduces the **1 cycle delay** that both write and read need.



currently the testbench is defining the `reg_addr` as `signal`(because as variable there was no difference), My findings:
From INCR onwards the `sram_wrddata` is showing perfect writing but
During READ:
the `sram_rdata` is only displaying `aa00_0000 bb00_0001 dd00_0003` (after using `reg_addr`). DURING WRITE: While checking the `sram_mem` i see that `aa00 0000 bb00 0001 cc00 0002` are written in crct snots(that

Show more

You are exactly right! The problem is clear from reading the `ahb_mem_bridge` code.

Root Cause Analysis

Problem 1: Count never resets between tests

Looking at your DUT code:

```
vhdl

-- INCR4 burst
when "011" =>
    if count = 4 then
        count := 0;    -- only resets when count REACHES 4
        flag := '1';
    else
        count := count + 1;
    end if;
```

The count only resets when it hits the **exact burst limit**. But since your TB runs test after test continuously, by the time WRAP4 arrives, count is already at 8 or 12 — it **never hits 4** so it never resets.

Problem 2: Count resets on NONSEQ but increments wrongly

Looking at NONSEQ state:

```
vhdl

when NONSEQ =>
    if v_burst = "000" then    -- SINGLE
        count := 0;
        flag := '1';
    else
        count := count + 1;    -- starts incrementing from whatever count was!
    end if;
```

Count should **reset to 0** on every NONSEQ (start of new burst) but instead it just keeps incrementing.

Problem 3: v_addr increment using count

```
vhdl
```

```
v_addr := std_logic_vector(unsigned(v_addr) +
                             to_unsigned(count, v_addr'length));
```

This adds `count` to address instead of adding a **fixed increment of 4** (for 32-bit word). So as count grows, addresses jump by wrong amounts.

The Fixes

Fix 1: Reset count to 0 on every NONSEQ

In the `when NONSEQ =>` section:

```
vhdl

when NONSEQ =>
    if v_we = '1' then
        inter_we <= '0';
        inter_oe <= '1';
        inter_ce <= '0';
    else
        inter_we <= '1';
        inter_oe <= '0';
        inter_ce <= '0';
    end if;
    -- FIX: always reset count at start of new burst
    if v_burst = "000" then
        count := 0;
        flag := '1';
    else
        count := 0;    -- ← CHANGED from count+1 to 0
    end if;
```

Fix 2: Fix address increment in SEQ — use fixed +4 not count

For every burst type in `when SEQ =>`, change:

```
vhdl

-- WRONG: adds count which keeps growing
v_addr := std_logic_vector(unsigned(v_addr) +
                             to_unsigned(count, v_addr'length));

-- FIXED: always add 4 bytes (32-bit word) then increment count
count := count + 1;
```



```
v_addr := std_logic_vector(unsigned(v_addr) +
                             to_unsigned(4, v_addr'length));
```

Fix 3: Fix WRAP burst boundary check

For WRAP4, WRAP8, WRAP16 the address wrapping logic also needs fixing.
Currently:

```
vhdl
-- WRONG: adds count to addr
v_addr := std_logic_vector(unsigned(v_addr) +
                             to_unsigned(count, v_addr'length));
```

Should be:

```
vhdl
-- FIXED: wrap correctly
-- For WRAP4 word: boundary = 16 bytes = 0x10
-- For WRAP8 word: boundary = 32 bytes = 0x20
-- For WRAP16 word: boundary = 64 bytes = 0x40
count := count + 1;
v_addr := std_logic_vector(
    (unsigned(v_addr) and not to_unsigned(boundary-1, 32)) or
    ((unsigned(v_addr) + 4) and to_unsigned(boundary-1, 32))
);
```

Fixed SEQ Section of ahb_mem_bridge

Replace the entire `when SEQ =>` case with this:

```
vhdl
when SEQ =>
    case v_burst is

        -- SINGLE burst (shouldn't get SEQ but handle it)
        when "000" =>
            inter_we <= '1';
            inter_oe <= '1';
            inter_ce <= '1';

        -- INCR (undefined length, 4 beats in our TB)
        when "001" =>
```

```
count := count + 1;
-- Fixed +4 byte increment
v_addr := std_logic_vector(
    unsigned(v_addr) + to_unsigned(4, v_addr'length));
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;

-- WRAP4: 4 beats, wraps at 16-byte boundary
when "010" =>
    if count = 3 then      -- 3 because count started at 0 on NONSEQ
        count := 0;
        flag := '1';
    else
        count := count + 1;
    end if;
-- Wrap within 16-byte boundary
v_addr := std_logic_vector(
    (unsigned(v_addr) and not to_unsigned(15, v_addr'length)) or
    ((unsigned(v_addr) + 4) and to_unsigned(15, v_addr'length)));
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;

-- INCR4: exactly 4 beats
when "011" =>
    if count = 3 then
        count := 0;
        flag := '1';
    else
        count := count + 1;
    end if;
v_addr := std_logic_vector(
    unsigned(v_addr) + to_unsigned(4, v_addr'length));
```

```
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;

-- WRAP8: 8 beats, wraps at 32-byte boundary
when "100" =>
    if count = 7 then
        count := 0;
        flag := '1';
    else
        count := count + 1;
    end if;
-- Wrap within 32-byte boundary
v_addr := std_logic_vector(
    (unsigned(v_addr) and not to_unsigned(31, v_addr'length)) or
    ((unsigned(v_addr) + 4) and to_unsigned(31, v_addr'length)));
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;

-- INCR8: exactly 8 beats
when "101" =>
    if count = 7 then
        count := 0;
        flag := '1';
    else
        count := count + 1;
    end if;
v_addr := std_logic_vector(
    unsigned(v_addr) + to_unsigned(4, v_addr'length));
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
```

```
    else
        inter_we <= '1';
        inter_oe <= '0';
        inter_ce <= '0';
    end if;

-- WRAP16: 16 beats, wraps at 64-byte boundary
when "110" =>
    if count = 15 then
        count := 0;
        flag  := '1';
    else
        count := count + 1;
    end if;
-- Wrap within 64-byte boundary
v_addr := std_logic_vector(
    (unsigned(v_addr) and not to_unsigned(63, v_addr'length)) or
    ((unsigned(v_addr) + 4) and to_unsigned(63, v_addr'length)));
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;

-- INCR16: exactly 16 beats
when "111" =>
    if count = 15 then
        count := 0;
        flag  := '1';
    else
        count := count + 1;
    end if;
v_addr := std_logic_vector(
    unsigned(v_addr) + to_unsigned(4, v_addr'length));
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;
```

```
        end if;

    when others =>
        inter_we <= '1';
        inter_oe <= '1';
        inter_ce <= '1';

    end case;
```

Also Fix NONSEQ Section

Replace `when NONSEQ =>` with:

```
vhdl

when NONSEQ =>
    if v_we = '1' then
        inter_we <= '0';
        inter_oe <= '1';
        inter_ce <= '0';
    else
        inter_we <= '1';
        inter_oe <= '0';
        inter_ce <= '0';
    end if;
    -- FIX: reset count to 0 for every new burst
    if v_burst = "000" then
        count := 0;
        flag  := '1';
    else
        count := 0;    -- reset here, SEQ will start incrementing from 1
    end if;
```

Summary of All Changes

What	Old (wrong)	Fixed
Count on NONSEQ	<code>count + 1</code>	<code>0</code> — reset every new burst
Address increment in SEQ	<code>+ count</code> (grows!)	<code>+ 4</code> (fixed word size)

What	Old (wrong)	Fixed
WRAP boundary check	<code>count = 4/8/16</code> (never hit after first test)	<code>count = 3/7/15</code> (0-indexed, resets correctly)
INCR count reset	Never resets	Doesn't need to — IDLE handles it

why have u used that `v_addr` equation for the WRAPs and also for the other burst types: why not use `v_addr := std_logic_vector(unsigned(v_addr) + to_unsigned(count*4, v_addr'length));`..doesn't multiplying the count make sense?

Great question! Let me explain both.

Why NOT `count * 4` for INCR/INCR4/INCR8/INCR16

`count * 4` would give you the **offset from start**, not the **next address**.

Start addr = 0x100, count starts at 0

```
count=0: v_addr = 0x100 + (0*4) = 0x100  ← beat 0 (NONSEQ handles this)
count=1: v_addr = 0x100 + (1*4) = 0x104  ← correct
count=2: v_addr = 0x104 + (2*4) = 0x10C  ← WRONG! should be 0x108
count=3: v_addr = 0x10C + (3*4) = 0x118  ← WRONG! should be 0x10C
```

Because `v_addr` is **already updated** each beat. You're adding `count*4` to an already-incremented address. It compounds.

The correct approach is just `+ 4` each beat:

```
v_addr = 0x100 → 0x104 → 0x108 → 0x10C  ✓
```

Why the WRAP equation for WRAPs

For WRAP4 starting at `0x118`:

- 16-byte boundary mask = `0x00F`
- Upper bits (fixed) = `0x110`
- Lower bits (wrap) = `0x008`

```
Beat 0: 0x118 → lower bits = 0x8
Beat 1: 0x11C → lower bits = 0xC
Beat 2: 0x110 → lower bits = 0x0 ← WRAPS here!
Beat 3: 0x114 → lower bits = 0x4
```

The equation:

```
vhdl
v_addr := (upper bits fixed) OR (lower bits increment and wrap)
         = (v_addr AND NOT 0xF) OR ((v_addr + 4) AND 0xF)
```

If you used `+ 4` directly:

```
0x118 + 4 = 0x11C ✓
0x11C + 4 = 0x120 × should wrap to 0x110!
```

It crosses the boundary instead of wrapping. The equation **keeps upper bits frozen** and only rolls over the lower bits — that's what wrapping means.

Bottom Line

Burst type	Why <code>+4</code>	Why WRAP equation
INCR/INCR4/8/16	Address just goes up by 4 each beat, simple	Not needed
WRAP4/8/16	<code>+4</code> alone crosses boundary	Need to freeze upper bits and wrap lower bits
<code>count*4</code>	Wrong — adds to already-updated address	Wrong — same problem

why have u taken the v_addr assignment outside the if loop in seq?

Good catch! Looking at the original DUT code in your PDF:

```
vhdl
```

```
when SEQ =>
  case v_burst is
    when "001" =>    -- INCR
      count := count + 1;
      v_addr := std_logic_vector(unsigned(v_addr) +
                                     to_unsigned(count, v_addr'length)); -- inside case
      if v_we = '1' then
        inter_we <= '0';
        ...
```

The `v_addr` update is **inside the case statement** but **outside the `if v_we` block**. That part is correct — address should increment **regardless of whether it's a read or write**.

In My Fixed Code

I kept `v_addr` outside the `if v_we` block but **inside the burst case**:

```
vhdl

when "011" =>    -- INCR4
  if count = 3 then
    count := 0;
    flag  := '1';
  else
    count := count + 1;
  end if;
v_addr := ...;      -- ← here, outside if v_we but inside "011" case
if v_we = '1' then  -- only control signals depend on read/write
  inter_we <= '0';
  ...
else
  inter_we <= '1';
  ...
end if;
```

This is correct because:

Signal	Depends on read/write?	Where updated
<code>v_addr</code>	NO — always increments	Outside <code>if v_we</code>
<code>inter_we</code>	YES	Inside <code>if v_we</code>
<code>inter_oe</code>	YES	Inside <code>if v_we</code>

Signal	Depends on read/write?	Where updated
<code>inter_ce</code>	NO — always active	Could be outside but kept inside for clarity

So it was intentional and correct. Was there a specific place you thought it looked wrong?

No i'm asking why u kept it outside the if count=3 then THAT if loop. Also explain to me how the HADDR was showing the crct address seq but not these sram_addr and other intermediate ones

Why v_addr is Outside the `if count=3` block

Good catch! Let me explain.

```

vhdl

when "011" =>    -- INCR4
    if count = 3 then
        count := 0;
        flag  := '1';
        -- NO v_addr update here -- last beat, no next address needed
    else
        count := count + 1;
    end if;
    v_addr := ...;    -- ← I put it here, outside

```

This is actually **wrong** for the last beat! On the last beat (`count=3`), we are setting `flag='1'` to signal burst complete. We **don't need** to calculate the next address because there is no next beat. But I still update `v_addr` unnecessarily.

The correct way matching your original DUT style:

```

vhdl

when "011" =>    -- INCR4
    if count = 3 then
        count := 0;
        flag  := '1';
        -- no v_addr update needed
    else
        count := count + 1;
    end if;

```

```

v_addr := std_logic_vector(    -- ← INSIDE else, only when not last beat
    unsigned(v_addr) + to_unsigned(4, v_addr'length));
end if;
if v_we = '1' then
    inter_we <= '0';
    inter_oe <= '1';
    inter_ce <= '0';
else
    inter_we <= '1';
    inter_oe <= '0';
    inter_ce <= '0';
end if;

```

Why HADDR showed correct sequence but not sram_addr

This is the pipeline issue again. Look at the signal flow:

TB drives:	DUT AHB Master Model:	DUT SRAM outputs:
haddr=0x100 → htrans=NONSEQ	v_addr=0x100 → curr_st=NONSEQ	inter_addr=0x100 sram_addr=0x100
haddr=0x104 → htrans=SEQ	v_addr=0x104 → curr_st=SEQ	inter_addr=0x104 sram_addr=0x104

They look the same. So why does `haddr` show correct but `sram_addr` doesn't?

Because haddr comes from the TESTBENCH

TB AHB master model calculates address independently:
`next_addr = cur_addr + 4` ← simple, correct every time
`haddr` follows this perfectly

But sram_addr comes from DUT's internal v_addr

DUT calculates `v_addr` using count:
`v_addr = v_addr + count` ← count is wrong/not reset!
 So `sram_addr` = wrong value

They are two completely separate address calculations:

Signal	Who calculates it	How
<code>haddr</code>	Testbench AHB master model	<code>cur_addr + 4</code> simple increment
<code>sram_addr</code>	DUT <code>ahb_mem_bridge</code> internal	<code>v_addr + count</code> — broken
<code>inter_addr</code>	DUT internal	same as <code>v_addr</code> — broken

So the TB was driving correct `haddr` into the DUT, but the DUT was **independently recalculating its own address** using `count` which was wrong. That's why you saw:

```
haddr:      0x100  0x104  0x108  0x10C  ← TB calculates, always correct
sram_addr:  0x100  0x106  0x10E  0x118  ← DUT calculates using count, wrong
```

The fix (using `+4` instead of `+count`) makes the DUT's internal address match what the TB is driving on `haddr`.

will fixing this address issue fix my writing it in wrong mem spot and the reading part?

Yes! Both problems come from the same root cause — wrong `v_addr` calculation.

Writing to Wrong Spot

```
haddr:      0x100  0x104  0x108  0x10C
v_addr:      0x100  0x106  0x10E  0x118  ← wrong because of count
```

```
sram_addr = inter_addr = v_addr(ABITS-1 downto 0)
```

So data gets written to:

```
0xAA000000 → mem[0x100] ← correct (NONSEQ, count=0)
0xBB000001 → mem[0x106] ← wrong! should be mem[0x104]
0xCC000002 → mem[0x10E] ← wrong! should be mem[0x108]
0xDD000003 → mem[0x118] ← wrong! should be mem[0x10C]
```

After fix with `+4`:

```
0xAA000000 → mem[0x100]  ✓
0xBB000001 → mem[0x104]  ✓
0xCC000002 → mem[0x108]  ✓
0xDD000003 → mem[0x10C]  ✓
```

Reading from Wrong Spot

Same issue — during read, `sram_addr` is wrong so the SRAM model looks up the wrong memory location:

```
Read beat 1: sram_addr=0x106 → mem[0x106] = 0  ← nothing written there!
Read beat 2: sram_addr=0x10E → mem[0x10E] = 0  ← nothing written there!
```

After fix:

```
Read beat 1: sram_addr=0x104 → mem[0x104] = 0xBB000001  ✓
Read beat 2: sram_addr=0x108 → mem[0x108] = 0xCC000002  ✓
```

Also the sram_rdata Sequence You Saw

```
During WRITE: 100 104 10b 10c ...  ← jumping wrongly
During READ:  100 104 113 114 ...  ← completely off
```

Both caused by wrong `v_addr`. After fix both will show:

```
During WRITE: 100 104 108 10C  ✓
During READ:  100 104 108 10C  ✓
```

Bottom Line

Problem	Root cause	Fixed by
Data written to wrong mem location	<code>v_addr = v_addr + count</code>	<code>v_addr = v_addr + 4</code>
Read returning 0 or wrong data	Reading wrong address	Same fix
<code>sram_rdata</code> showing wrong sequence	Wrong <code>sram_addr</code> fed to SRAM model	Same fix

Problem	Root cause	Fixed by
WRAP tests failing	Count never resets + wrong addr	Count reset on NONSEQ + wrap equation

One fix — `v_addr + 4` instead of `v_addr + count` — cascades and fixes all three problems.

Tell me how to calculate the sram_mem index for a given address

Simple formula:

`index = address / 4`

Because each memory location stores **32 bits (4 bytes)**, so every 4 bytes of address = 1 memory slot.

Examples

Address	Index	Calculation
<code>0x100</code>	64	<code>0x100 / 4 = 256/4 = 64</code>
<code>0x104</code>	65	<code>0x104 / 4 = 260/4 = 65</code>
<code>0x108</code>	66	<code>0x108 / 4 = 264/4 = 66</code>
<code>0x10C</code>	67	<code>0x10C / 4 = 268/4 = 67</code>

In VHDL That's What This Does

```
vhdl
sram_mem(to_integer(unsigned(sram_addr(11 downto 2))))
```

Taking `sram_addr(11 downto 2)` means **dropping the bottom 2 bits** which is the same as dividing by 4:

`0x100 = 0001 0000 0000`
 `^^^^^^^^^^^^^^`

```
bits 11 downto 2 = 0100 0000 = 64  ✓
```

```
0x104 = 0001 0000 0100
```

```
bits 11 downto 2 = 0100 0001 = 65  ✓
```

```
0x108 = 0001 0000 1000
```

```
bits 11 downto 2 = 0100 0010 = 66  ✓
```

```
0x10C = 0001 0000 1100
```

```
bits 11 downto 2 = 0100 0011 = 67  ✓
```

So Your Earlier Observation Makes Sense Now

You said `0xDD000003` was written to index **68** instead of **67**. That means it was written to address `0x110` instead of `0x10C` — exactly a `+4` error caused by the wrong `count` calculation in `v_addr`.

